ATmega32A Module
ADC (Analog-to-Digital Converter)

Ali Sahafi

1 Theory

The world outside the microcontroller is *analog*: a voltage can have any value, for example 1.37 V. The CPU can only work with numbers. An ADC (*Analog-to-Digital Converter*) measures a voltage and gives you a number.

Resolution and reference. The ATmega32A has a **10-bit** ADC. So the result is a number from 0 to $2^{10} - 1 = 1023$. The *reference voltage* V_{REF} is the largest voltage the ADC can measure. On our board $V_{\text{REF}} = 5\,\text{V}$. One step of the result (1 LSB) is

$$1\,\text{LSB} = \frac{V_{\text{REF}}}{1024} = \frac{5\,\text{V}}{1024} \approx 4.9\,\text{mV}$$

and the result for an input voltage V_{IN} is

$$\text{value} = \frac{V_{\text{IN}} \cdot 1024}{V_{\text{REF}}} \qquad V_{\text{IN}} = \frac{\text{value} \cdot V_{\text{REF}}}{1024}$$

Input voltage	10-bit value	8-bit value
0 V	0	0
1.25 V	256	64
2.5 V	512	128
3.75 V	768	192
5 V	1023	255

If 8 bits are enough (for example for 8 LEDs or for PWM), we use only the upper 8 bits of the result: 0 to 255.

Channels. There is only one ADC, but it has a multiplexer with 8 inputs: **ADC0**–**ADC7** on the pins **PA0**–**PA7**. You choose the channel for each measurement.

Conversion time. The ADC needs its own clock between 50 and 200 kHz for the full 10-bit accuracy. It is made from the CPU clock with a prescaler. One conversion takes 13 ADC clock cycles: at 125 kHz this is about 104 μs.

2 How to use the driver

The driver gives you one object: `ADC`. First you switch the ADC on one time with `ADC.enable()`. This does not measure anything yet. Then you call `ADC.read()` every time you need a new value: each call makes one measurement. When you do not need the ADC any more, `ADC.disable()` switches it off again, so it uses no power.

Functions

Function	What it does
`ADC.enable()`	Switch the ADC on (bit `ADEN` = *ADC Enable*). It does not measure. Reference = AREF (5 V on our board). The driver chooses the ADC clock from `F_CPU` (8 MHz / 64 = 125 kHz)
`ADC.disable()`	Switch the ADC off (`ADEN` = 0). It then uses no power. While it is off, every read returns 0
`ADC.read(channel)`	Measure the voltage on `PA`*channel* (channel 0–7) and return 0–1023. It waits until the conversion is finished (about 104 μs)
`ADC.read8(channel)`	The same, but only the upper 8 bits: 0–255. Good for LEDs and PWM
`ADC.readMillivolts(channel)`	The same, in millivolts: 0–4995 mV
With interrupts (see the interrupt lecture):	
`ADC.start(channel)`	Start one conversion and return at once (no waiting)
`ADC.onComplete(function)`	Call `function(value)` when a conversion is finished (value 0–1023)

The pin. `read()` makes the pin an *input* and switches its pull-up resistor *off*. A pull-up would change the voltage that you want to measure. So you do not need `GPIO.setDirection()` for an ADC pin.

Reference voltage on our board. On the course board the pins `VCC`, `AVCC` and `AREF` are all connected to +5 V. For this reason the driver always uses `AREF` as the reference. The internal 2.56 V reference must not be used on this board: it would be connected to the 5 V on the `AREF` pin.

Driver functions and registers

In the lecture slides we set the registers by hand. Each driver function does these steps for you:

Driver function	Registers
`ADC.enable()`	`ADMUX = 0;` (`REFS1:0` = 00: AREF; `ADLAR` = 0) `ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1);` (prescaler 64 at 8 MHz)
`ADC.disable()`	`ADCSRA &= ~(1<<ADEN);`
`ADC.read(0)`	`DDRA &= ~(1<<PA0); PORTA &= ~(1<<PA0);` (input, no pull-up) `ADMUX = (ADMUX & 0xE0) | 0;` (`MUX4:0` = channel) `ADCSRA |= (1<<ADSC); while (ADCSRA & (1<<ADSC));` `return ADCW;` (`ADCL` first, then `ADCH`)
`ADC.read8(0)`	`ADC.read(0) >> 2` (the same as `ADLAR` = 1 and reading only `ADCH`)
`ADC.readMillivolts(0)`	`ADC.read(0) * 5000 / 1024`

3 Example: a small voltmeter

Board hardware.

- **Potentiometer** R13 (10 kΩ) on `PA0` = **ADC0**. Turn it to change the voltage on `PA0` from 0 to 5 V.
- **8 LEDs (D0–D7) on PORTB**, active-low: `LOW` = ON. LED **D3** is on `PB3` = `OC0`, the PWM output of Timer0.
- **USB-UART** (FT232) on `PD0`/`PD1`: open a serial terminal with 9600 baud, 8N1 (see the UART handout).

Build and upload. `make adc`. Open the terminal and turn the potentiometer. You see for example `ADC = 759 (3706 mV)`.

```
/*
 * ADC Example -- a small voltmeter: read the potentiometer and send the
 * result to the PC two times per second.
 * Build & flash: make adc
 *
 * Open a serial terminal on the PC: 9600 baud, 8N1. Turn the potentiometer
 * and watch the numbers, for example:
 *    ADC = 759  (3706 mV)
 *
 * The potentiometer of the board is on PA0 (ADC channel 0).
 */

#define F_CPU 8000000UL
#include "../uart/uart.hpp"
#include "adc.hpp"

int main() {
  UART.begin(9600);
  ADC.enable(); // reference AREF = 5 V, ADC clock 125 kHz

  while (true) {
    uint16_t value = ADC.read(0);         // 0..1023
    uint16_t mv = ADC.readMillivolts(0);  // 0..4995 mV

    UART.print("ADC = ");
    UART.print(value);
    UART.print("  (");
    UART.print(mv);
    UART.println(" mV)");

    _delay_ms(500);
  }
}
```

4 Laboratory Tasks

For each task, make a new `.cpp` file in the main project folder (for example `Task1.cpp`). Start the file with `#define F_CPU 8000000UL`. Then include the drivers you need: `drivers/adc/adc.hpp`, `drivers/gpio/gpio.hpp`, `drivers/timer/timer.hpp`, `drivers/uart/uart.hpp`. Upload it with `make SRC=Task1.cpp`.

Task 1: Potentiometer on the LEDs (25 min)

Read the potentiometer (channel 0) and show the value on the 8 LEDs on `PORTB`. Use `ADC.read8(0)`, because there are only 8 LEDs. The LEDs are active-low, so write the inverted value: `GPIO.write(PORTB, ~value)`.

Turn the potentiometer from one end to the other. Which LEDs are on at the two ends? Which value do you read in the middle?

Task 2: Potentiometer Sets the LED Brightness (20 min)

Use the potentiometer to set the brightness of LED **D3** (`PB3` = `OC0`):

1. Start `Timer0` in `TIMER_FAST_PWM` mode with `PWM_INVERTING` (the LED is active-low), as in the timer lesson.
2. In the loop, read the potentiometer with `ADC.read8(0)` and write the value with `Timer0.setCompare()`.

Now the potentiometer works like a light dimmer.

Task 3: LM35 Thermometer on the UART (25 min)

The **LM35** is an analog temperature sensor. Its output voltage is **10 mV per °C**, starting at 0 mV for 0 °C:

$$T\,[^\circ\mathrm{C}] = \frac{V_{\mathrm{OUT}}\,[\mathrm{mV}]}{10} \qquad \text{for example} \quad 250\,\mathrm{mV} \;\rightarrow\; 25.0\,^\circ\mathrm{C}$$

Wiring. Hold the LM35 with the flat side (text) to you, pins down. From left to right:

LM35 pin	Name	Board
1 (left)	$+V_S$	+5 V
2 (middle)	V_{OUT}	`IO_PA1` = `PA1` = **ADC1**
3 (right)	GND	GND

Check the wiring two times before you switch on: if $+V_S$ and GND are swapped, the sensor gets hot. `PA0` is used by the potentiometer, so we use channel 1.

1. Measure the sensor voltage with `ADC.readMillivolts(1)`.
2. Calculate the temperature: whole degrees `mv / 10` and the tenths `mv % 10`.
3. Send it to the PC with the UART (9600 baud) one time per second, for example `T = 23.4 C`.
4. Hold the sensor between your fingers. How fast does the temperature go up? How high does it go?

Assignment: Find the Cross Voltage

The pin `PA0` can be read in two ways: as an *analog* value with the ADC, and as a *digital* value (0 or 1) with `GPIO.read(PINA, PA0)`. At some input voltage the digital value changes from 0 to 1. Find this *cross voltage*.

1. In the loop, read both values of `PA0`: `ADC.readMillivolts(0)` and `GPIO.read(PINA, PA0)`.
2. Send both to the PC with the UART (Lesson 6), for example `3706 mV -> 1`, about 10 times per second.
3. Turn the potentiometer *slowly* from 0 V up to 5 V. At which voltage does the digital value change from 0 to 1?
4. Now turn it slowly back down. At which voltage does it change from 1 to 0? Is it the same voltage? Compare your results with the input voltages V_{IL} and V_{IH} in the datasheet (*Electrical Characteristics*).